

FCN - Sviluppo reflectors

I riflettori “classici” che entrano nei metodi di fattorizzazione sono essenzialmente tre famiglie: trasformazioni elementari di Gauss (che non sono ortogonali), riflettori di Householder (riflessioni ortogonali di rango 1) e rotazioni di Givens (ortogonali a supporto 2×2).¹²³⁴⁵⁶

Schematizzazione unificata con indicazioni d’uso e due esempi concreti (uno Householder, uno Givens).

1. Matrici elementari in forma unificata

Una buona prospettiva astratta è: molte trasformazioni usate in algoritmi di eliminazione o di ortogonalizzazione sono matrici del tipo

$$M = I + \sigma uv^T,$$

con $u, v \in \mathbb{R}^n$ e $\sigma \in \mathbb{R}$, da cui segue che $M - I$ ha rango al più 1.⁷

- Nel caso Householder, $M = I - 2vv^T$ con $\|v\|_2 = 1$, quindi $\sigma = -2$ e $u = v$.⁸⁹¹⁰
- Molte matrici di Gauss che realizzano “una singola mossa” sono della forma $I + \sigma e_i e_j^T$ con $i \neq j$, quindi ancora rango 1.¹¹¹²

Le rotazioni di Givens sono invece matrici identità modificate su un blocco 2×2 ortogonale; la struttura è ancora molto localizzata, ma il rango di $G - I$ è tipicamente 2, non 1.¹³¹⁴

2. Matrici elementari di Gauss

Queste sono le matrici che rappresentano algebricamente le tre operazioni elementari sulle righe (o colonne) usate nell’eliminazione di Gauss e nelle fattorizzazioni LU.¹⁵¹⁶¹⁷¹⁸

Operazioni elementari sulle righe (versione “righe”):

- Scambio di due righe i e j : matrice di permutazione P_{ij} , ottenuta scambiando le righe i e j dell’identità.¹⁹²⁰
- Moltiplicazione della riga i per uno scalare non nullo α : matrice diagonale D con $D_{ii} = \alpha$, altri diagonal = 1.²¹²²
- Somma di un multiplo della riga j alla riga i :

$$E = I + \sigma e_i e_j^T, \quad \sigma \in \mathbb{R},$$

che aggiunge σ volte la riga j alla riga i .²³²⁴²⁵

Uso tipico:

- Eliminazione di Gauss / Gauss con pivoting: si applica una successione di matrici elementari di tipo “somma di multipli di righe” (più eventualmente permutazioni e scalamenti) per introdurre zeri sotto la diagonale della matrice; la fattorizzazione LU si può leggere come prodotto di queste matrici (o delle loro inverse).²⁶²⁷²⁸

Pro e contro, computazionalmente:

- Pro: struttura estrema (una sola riga non banale, o una coppia di righe scambiate), costo $O(n)$ per l’applicazione ad un vettore.²⁹³⁰
 - Contro: non sono ortogonali (tranne permutazioni e scalamenti con modulo 1), quindi possono amplificare gli errori di arrotondamento; per problemi di least squares e fattorizzazioni ortogonali si preferiscono Householder o Givens.³¹³²
-

3. Riflettori di Householder

Definizione:

Una trasformazione di Householder in $\mathbb{R}^n$ ha la forma

$$H = I - 2vv^T,$$

con $v \in \mathbb{R}^n$, $\|v\|_2 = 1$.³³³⁴³⁵

Proprietà:

- H è simmetrica e ortogonale: $H^T = H$, $H^T H = I$.³⁶³⁷³⁸
- Geometricamente è una riflessione rispetto all'iperpiano ortogonale a v .³⁹⁴⁰
- Moltiplicare un vettore x per H costa $O(n)$: si calcola prima $\alpha = v^T x$, poi $Hx = x - 2\alpha v$.⁴¹⁴²

Uso tipico:

- Fattorizzazione QR “globale”: dato $A \in \mathbb{R}^{m \times n}$, si costruisce una sequenza di riflettori $H_1, \dots, H_k$ tali che

$$H_k \cdots H_1 A = R,$$

con R triangolare superiore, e $Q = H_1^T \cdots H_k^T$ è ortogonale.⁴³⁴⁴⁴⁵ - Riduzione a forma bidiagonale (per SVD), Hessenberg, ecc.: sempre giocando con riflettori applicati da sinistra e/o destra.⁴⁶⁴⁷

Scelta di v per “zerare” una colonna:

- Dato $x \in \mathbb{R}^m$ (per esempio, la parte sotto-diagonale di una colonna), si vuole un H che mandi x in uno scalare times e_1 : $Hx = \pm\|x\|_2 e_1$.⁴⁸⁴⁹⁵⁰
- Un costruito numericamente stabile è:
 - $\alpha = \text{sign}(x_1)\|x\|_2$.
 - $u = x + \alpha e_1$.
 - $v = u/\|u\|_2$.
 - Allora $H = I - 2vv^T$.⁵¹⁵²⁵³

4. Rotazioni di Givens

Definizione:

Una rotazione di Givens agisce su un piano generato da due coordinate i e j ; in forma 2×2 :

$$G_2 = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}, \quad c^2 + s^2 = 1.$$

⁵⁴⁵⁵

In dimensione n , la Givens $\mathcal{G}_{ij}$ è l'identità tranne nel blocco 2×2 sugli indici i, j , dove compare G_2 .⁵⁶⁵⁷

Uso tipico:

- Fattorizzazione QR “locale”: si eliminano gli elementi subdiagonali uno per volta applicando una successione di rotazioni di Givens che agiscono su coppie di righe.⁵⁸⁵⁹
- Molto usate quando la matrice è sparsa o quando si vogliono operazioni “locali” (per esempio aggiornamenti di QR quando si aggiunge una riga/colonna).⁶⁰⁶¹

Scelta di c, s per annullare una componente:

- Dato un vettore $(x, y)^T$, si vogliono c, s tali che

$$\begin{bmatrix} c & -s \\ s & c \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix}.$$

- Una scelta stabile è

$$r = \sqrt{x^2 + y^2}, \quad c = x/r, \quad s = -y/r.$$

Pro e contro (rispetto a Householder):

- Pro: agiscono su pochi elementi; ideali per aggiornamenti a bassa complessità e per matrici sparse, perché non “riempiono” intere colonne.⁶³⁶⁴
- Contro: per trasformazioni dense (QR completo) costano più floating-point operations rispetto a Householder, dato che servono molte rotazioni per annullare una colonna completa.⁶⁵⁶⁶

5. Esempi numerici (schemi d'uso)

Esempio 1: Householder che annulla la coda di un vettore

Dato

$$x = \begin{bmatrix} 4 \\ 3 \\ 0 \end{bmatrix}, \quad x \in \mathbb{R}^3,$$

costruiamo un Householder H che mandi x in $\pm\|x\|_2 e_1$.

1. Norma: $\|x\|_2 = \sqrt{4^2 + 3^2 + 0^2} = 5$.
2. Scegli $\alpha = \text{sign}(4) \cdot 5 = 5$.
3. $u = x + \alpha e_1 = \begin{bmatrix} 4+5 \\ 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 9 \\ 3 \\ 0 \end{bmatrix}$.
4. $\|u\|_2 = \sqrt{9^2 + 3^2} = \sqrt{81 + 9} = \sqrt{90} = 3\sqrt{10}$.
5. $v = u/\|u\|_2 = \frac{1}{3\sqrt{10}} \begin{bmatrix} 9 \\ 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 3/\sqrt{10} \\ 1/\sqrt{10} \\ 0 \end{bmatrix}$.
6. $H = I - 2vv^T$.

Calcoliamo vv^T :

$$vv^T = \begin{bmatrix} 3/\sqrt{10} \\ 1/\sqrt{10} \\ 0 \end{bmatrix} \begin{bmatrix} 3/\sqrt{10} & 1/\sqrt{10} & 0 \end{bmatrix} = \frac{1}{10} \begin{bmatrix} 9 & 3 & 0 \\ 3 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Allora

$$H = I - \frac{2}{10} \begin{bmatrix} 9 & 3 & 0 \\ 3 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1-1.8 & -0.6 & 0 \\ -0.6 & 1-0.2 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -0.8 & -0.6 & 0 \\ -0.6 & 0.8 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Controllo:

$$Hx = \begin{bmatrix} -0.8 & -0.6 & 0 \\ -0.6 & 0.8 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 3 \\ 0 \end{bmatrix} = \begin{bmatrix} -5 \\ 0 \\ 0 \end{bmatrix} = -5e_1,$$

come previsto.⁶⁷⁶⁸⁶⁹

Uso in QR: questo H sarebbe il primo riflettore applicato alla prima colonna di una matrice 3×3 per introdurre zeri sotto la diagonale; i riflettori successivi operano sui subblocchi inferiori destri.⁷⁰⁷¹⁷²

Esempio 2: Givens che annulla una singola componente

Vogliamo annullare la seconda componente del vettore

$$z = \begin{bmatrix} 3 \\ 4 \end{bmatrix},$$

mediante una rotazione di Givens 2×2 .⁷³

1. $r = \sqrt{3^2 + 4^2} = 5$.
2. $c = x/r = 3/5$.
3. $s = -y/r = -4/5$.

La rotazione

$$G_2 = \begin{bmatrix} 3/5 & 4/5 \\ -4/5 & 3/5 \end{bmatrix}$$

(dove ho usato la convenzione con il segno su s compatibile con l'annullamento) soddisfa:

$$G_2 \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}.$$

Se inseriamo questo blocco in una matrice identità 3×3 , operante su righe 2 e 3, otteniamo una rotazione di Givens $\mathcal{G}_{23}$ in $\mathbb{R}^3$ che annulla l'elemento in posizione (3,1) di una matrice, agendo sulle righe 2 e 3.⁷⁴⁷⁵

6. Altri trasformatrici “simili” a riflettori

Oltre a Gauss, Householder e Givens, nel design di algoritmi entrano altre trasformazioni “elementari” affini:

- Riflettori generalizzati di rango 1: $M = I - \sigma uu^T$ con $\sigma \neq 2$ possono essere usati per aggiornamenti rank-1 in contesti simili a Householder (non necessariamente ortogonali).⁷⁶⁷⁷
- Trasformazioni unitariamente equivalenti: nelle versioni complesse, riflettori di Householder diventano $H = I - 2 \frac{uv^*}{v^*v}$, con v^* coniugato trasposto; Givens diventano rotazioni complesse.⁷⁸⁷⁹⁸⁰

In pratica, la “triade” **Gauss–Householder–Givens** copre la maggior parte dei costrutti elementari usati in fattorizzazioni LU/QR, riduzioni a forma canonica (Hessenberg, tridiagonale, bidiagonale) e aggiornamenti strutturati.⁸¹⁸²⁸³⁸⁴

Strategia didattica

Per riflettori e trasformazioni elementari, **prima separare bene i comportamenti numerici**, solo dopo costruire un'interfaccia comune, perché Gauss, Householder e Givens hanno struttura, costo e uso algoritmico diversi anche se tutti agiscono come operatori elementari su righe/colonne o sottospazi molto piccoli.⁸⁵⁸⁶⁸⁷⁸⁸

La progressione più naturale è: Householder per il caso “riflettore puro”, Givens per la trasformazione locale su due componenti, Gauss per la mossa eliminativa non ortogonale; così lo studente vede subito differenze geometriche, algebriche e numeriche prima della sintesi API.⁸⁹⁹⁰⁹¹

In pratica:

- Householder: annulla in un colpo tutta la coda di un vettore o di una colonna, ed è il mattone standard per QR dense e riduzioni ortogonali.⁹²⁹³⁹⁴
- Givens: annulla una sola componente alla volta, ma è molto adatto a aggiornamenti locali e matrici sparse.⁹⁵⁹⁶⁹⁷
- Gauss: è il generatore naturale per eliminazione/LU, ma non preserva ortogonalità e quindi va tenuto concettualmente distinto dai riflettori ortogonali.⁹⁸⁹⁹¹⁰⁰

Interfaccia prima dell'unificazione

Prima dell'interfaccia unica, conviene dare a ciascun trasformatore un piccolo contratto operativo comune: costruzione, applicazione a vettore, applicazione a matrice da sinistra/destra, eventuale inversione o trasposto.¹⁰¹¹⁰²¹⁰³

Uno schema FCN plausibile potrebbe essere:

- `build(x, policy)` oppure `from_column_segment(...)`
- `apply_left(A, k0, k1, ...)`
- `apply_right(A, k0, k1, ...)`
- `apply(x)`
- `as_dense()` solo per test e didattica, non per uso interno prestazionale.¹⁰⁴¹⁰⁵¹⁰⁶

Questo ti permette di fare sperimentazione comparativa uniforme senza forzare subito una parametrizzazione artificiale comune.¹⁰⁷¹⁰⁸¹⁰⁹

Criterio di unificazione

Dopo la fase didattica, l'unificazione ha senso se l'interfaccia comune descrive **l'azione** e non la formula interna del trasformatore.¹¹⁰¹¹¹

Per esempio, l'astrazione giusta non è necessariamente “tutti sono $I + \sigma uv^T$ ”, perché questo descrive bene Gauss e Householder ma non Givens in modo naturale, dato che Givens vive meglio come operatore ortogonale con supporto 2×2 su indici (i, j) .¹¹²¹¹³ Molto meglio una base concettuale del tipo:

- `Transformation`
- `OrthogonalTransformation`
- `EliminationTransformation`

e poi specializzazioni:

- `HouseholderReflector`
- `GivensRotation`
- `GaussEliminationStep`¹¹⁴¹¹⁵¹¹⁶

Minimo insieme di varianti

Per minimizzare le varianti, suggerirei di fissare subito poche scelte non negoziabili:

- Convenzione unica di segno per Householder, per evitare cancellazione numerica nella costruzione del vettore riflettente.¹¹⁷¹¹⁸¹¹⁹
- Givens sempre costruita da coppia (x, y) con routine stabile per c, s .¹²⁰
- Gauss distinta in versione con e senza pivoting, ma con la stessa interfaccia esterna.¹²¹¹²²
- Rappresentazione implicita come default; esplicita solo per debug, test unitari e note didattiche.¹²³¹²⁴

Così l'unificazione finale non cancella le differenze matematiche, ma riduce il rumore implementativo.¹²⁵¹²⁶

Proposta operativa

Io partirei con questo ordine di sviluppo:

1. `HouseholderReflector`, come prototipo di trasformatore ortogonale implicito e “globale” su una coda.¹²⁷¹²⁸
2. `GivensRotation`, come prototipo di trasformatore ortogonale locale.¹²⁹¹³⁰
3. `GaussEliminationStep`, come trasformatore eliminativo non ortogonale.¹³¹¹³²
4. Test comparativi comuni su: annullamento, costo operativo, stabilità, fill-in e uso in QR/LU.¹³³¹³⁴¹³⁵¹³⁶
5. Solo a quel punto, una facciata unificata `apply_left/apply_right/apply/vector_action`.¹³⁷¹³⁸

Questa sequenza è coerente con un'impostazione didattica-sperimentale e ti lascia aperta una futura unificazione elegante senza “violentare” Givens dentro una forma troppo pensata per Householder.¹³⁹¹⁴⁰

Se vuoi, nel passo successivo posso proporti direttamente una **bozza C++ dell'interfaccia FCN** a tre classi distinte ma già predisposte per la successiva unificazione.¹⁴¹¹⁴²¹⁴³¹⁴⁴¹⁴⁵¹⁴⁶¹⁴⁷¹⁴⁸

Il piano che ho in mente separa esplicitamente e didatticamente le due fasi per ciascun trasformatore: **costruzione** (scelta di parametri u, v , o v solo, o c, s per Givens, o moltiplicatori di Gauss in funzione del target), e **applicazione** (uso ripetuto, possibilmente ottimizzato, di quella trasformazione su vettori/matrici).¹⁴⁹¹⁵⁰¹⁵¹¹⁵²¹⁵³¹⁵⁴

comune e poi il dettaglio per Householder, Givens e Gauss.

1. Schema concettuale comune e dettagli per Householder, Givens e Gauss.

Per ogni trasformazione T avremo due famiglie di operazioni ben distinte:

1. Fase di costruzione (design):

- Input: “target” (ad esempio un vettore colonna da trasformare, o una coppia di componenti da azzerare, o una colonna di matrice in eliminazione).
- Output: un set minimo di parametri che rappresentano T in forma implicita (es. v per Householder, c, s per Givens, moltiplicatori per Gauss).¹⁵⁵¹⁵⁶¹⁵⁷¹⁵⁸

2. Fase di applicazione (use):

- Operazioni: `apply(x)` (vettore), `apply_left(A, range)`, `apply_right(A, range)`.
- Tutto scritto in modo da rendere evidente la formula: proiezioni scalari, aggiornamento outer-product, ecc., senza “nascondere” la struttura sotto un generico `operator*` black-box.¹⁵⁹¹⁶⁰¹⁶¹¹⁶²

In termini di interfaccia penserei proprio a due gruppi di metodi: `construct_*` vs `apply_*`, o un costruttore che fa solo la parte “geometrica” e metodi d’istanza che fanno la parte “algebrica”.¹⁶³¹⁶⁴¹⁶⁵¹⁶⁶

2. Householder: costruzione vs applicazione

Principio: un riflettore di Householder è $H = I - 2vv^T$ con $\|v\|_2 = 1$; la costruzione consiste nello scegliere v in funzione di un target x che vogliamo mandare su una direzione data (tipicamente e_1), l’applicazione consiste nel calcolare $Hx = x - 2(v^T x)v$.¹⁶⁷¹⁶⁸¹⁶⁹¹⁷⁰

Fase di costruzione (scelta di v)

Dato $x \in \mathbb{R}^m$ (ad esempio la sotto-colonna da riga k a m), si vuole H con $Hx = \pm\|x\|_2 e_1$ (nel sottospazio considerato).¹⁷¹¹⁷²¹⁷³¹⁷⁴

Algoritmo numericamente stabile:¹⁷⁵¹⁷⁶¹⁷⁷

1. $\alpha = \text{sign}(x_1)\|x\|_2$ (scelta del segno per evitare cancellazioni).¹⁷⁸¹⁷⁹
2. $u = x + \alpha e_1$.
3. $v = u/\|u\|_2$.
4. Memorizzi v (non H).¹⁸⁰¹⁸¹

Qui è importante la scelta del segno di α (analisi della cancellazione in $x_1 - \alpha$) e il fatto che H è una riflessione rispetto all’iperpiano ortogonale a v .¹⁸²¹⁸³¹⁸⁴¹⁸⁵

Fase di applicazione

Data la v scelta:

- Su vettore:
 1. $\beta = v^T x$.
 2. $y = x - 2\beta v$.¹⁸⁶¹⁸⁷¹⁸⁸
- Su matrice a sinistra (aggiorno alcune righe):
 1. $\beta = v^T A$ (riga o sottoblocco di righe).
 2. $A \leftarrow A - 2v\beta$.¹⁸⁹¹⁹⁰¹⁹¹

Tutto in forma implicita, con la formula scritta “in chiaro” nel codice; l’interfaccia può avere metodi separati `construct_from_column(...)` e `apply_left(...)`, quindi la distinzione è esplicita anche a livello didattico.¹⁹²¹⁹³¹⁹⁴

3. Givens: costruzione vs applicazione

Principio: una rotazione di Givens è un'ortogonale che agisce solo sul piano (i, j) , e serve a nullificare una singola componente y usando la coppia (x, y) .¹⁹⁵¹⁹⁶

Fase di costruzione (scelta di c, s)

Dato (x, y) , con x la componente che “sopravvive” e y quella da eliminare, si cercano c, s tali che

$$\begin{bmatrix} c & s \\ -s & c \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix}.$$

¹⁹⁷¹⁹⁸¹⁹⁹

Tipico algoritmo stabile:²⁰⁰²⁰¹

- $r = \sqrt{x^2 + y^2}$.
- $c = x/r, s = y/r$ (o variante con segni secondo la convenzione).

In codice, constructor o `construct(x,y)` che memorizza c, s e gli indici i, j su cui agire.²⁰²²⁰³²⁰⁴

Fase di applicazione

- Su vettore: aggiornare solo le componenti i, j :

$$\begin{aligned} t_i &= cx_i + sx_j, \\ t_j &= -sx_i + cx_j. \end{aligned}$$

²⁰⁵²⁰⁶

- Su matrice a sinistra: combinare solo le righe i, j . Su matrice a destra: combinare solo le colonne i, j .²⁰⁷²⁰⁸²⁰⁹

Ancora, la separazione è naturale: un metodo `construct(i,j,x,y)` produce i parametri, poi `apply_left(A)` implementa in chiaro la combinazione lineare di righe.²¹⁰²¹¹

4. Gauss: costruzione vs applicazione (elementary matrices)

Principio: l'eliminazione di Gauss è la costruzione di una sequenza di matrici elementari E_k che realizzano scambi, scalamenti, e aggiunge multipli di righe.²¹²²¹³²¹⁴²¹⁵

Per la singola mossa di eliminazione su una colonna:

Fase di costruzione (scelta dei moltiplicatori)

Dato un pivot a_{kk} e l'elemento a_{ik} da annullare, il moltiplicatore è

$$m_{ik} = a_{ik}/a_{kk}.$$

²¹⁶²¹⁷²¹⁸²¹⁹

La matrice elementare implicita è $E = I - m_{ik}e_i e_k^T$; in pratica memorizzi m_{ik} e gli indici i, k .²²⁰²²¹²²²

Fase di applicazione

- Su matrice: riga $i \leftarrow \text{riga } i - m_{ik} \cdot \text{riga } k$.²²³²²⁴²²⁵²²⁶
- Su vettore (per applicare l'operatore al lato destro in un sistema): stessa regola sulle componenti corrispondenti.²²⁷²²⁸

Anche qui è molto didattico lasciare a vista la relazione con la forma $E = I + \sigma e_i e_j^T$ per legare il tutto alle trasformazioni di rango 1.²²⁹²³⁰

5. Come riflettere tutto questo nell'interfaccia

Per mantenere la distinzione concettuale, ma prepararsi all'unificazione, io imposterei così:

- Ogni classe ha un **metodo di costruzione esplicito** (o costruttore statico) che usa **target + politica numerica**:
 - `HouseholderReflector::from_column_segment(x, policy_sign_choice)`
 - `GivensRotation::from_pair(i,j,x,y, policy_sign_choice)`
 - `GaussEliminationStep::from_pivot(a_kk, a_ik, pivot_policy)`²³¹²³²²³³²³⁴
- Ogni classe espone metodi di **applicazione ben visibili**:
 - `apply(Vector&) const`
 - `apply_left(Matrix&, Range rows, Range cols) const`
 - `apply_right(Matrix&, Range rows, Range cols) const`

in cui il codice implementa esplicitamente le formule che hai spiegato nella parte teorica.²³⁵²³⁶²³⁷²³⁸²³⁹

- In una fase successiva, potrai interfacciare queste classi a una gerarchia comune tipo `LinearOperator` / `OrthogonalOperator`, ma senza nascondere mai i metodi di costruzione specifici (che sono la parte “pedagogica”).²⁴⁰²⁴¹²⁴²

Modello OOP (esempio)

Scheletro C++ per un **HouseholderReflector** didattico, con le due fasi volutamente separate e le formule rese esplicite, sul modello “genera riflettore” / “applica riflettore” che usano anche LAPACK (DLARFG per la generazione, DLARF per l’applicazione).²⁴³²⁴⁴²⁴⁵²⁴⁶

L’idea è: la classe incapsula il vettore v (e, facoltativamente, $\tau = 2$ o l’equivalente $\tau = 2/(v^T v)$ nella versione “non normalizzata”), mentre i metodi `construct_from_segment` e `apply_*` esplicitano rispettivamente la fase geometrica (scelta di v) e la fase algebrica (calcolo $x - 2(v^T x)v$ o $A - 2v(v^T A)$).²⁴⁷²⁴⁸²⁴⁹

Scheletro di HouseholderReflector

Assumo:

- un container tipo `Vector/Matrix` già tuo (o `std::vector<double>` + funzioni helper),
- reali in doppia precisione,
- versione “classica”: vogliamo che il riflettore mandi un segmento di colonna in un multiplo di e_1 .²⁵⁰²⁵¹²⁵²²⁵³

```
// HouseholderReflector.hpp
#pragma once
#include <cstdint>
#include <vector>
#include <cmath>
#include <cassert>

class HouseholderReflector {
public:
    using index_type = std::size_t;
    using real_type = double;

    // Costruito vuoto, da riempire con construct_from_segment.
    HouseholderReflector() = default;

    // Costruttore da vettore v già normalizzato (uso avanzato/didattico).
    explicit HouseholderReflector(const std::vector<real_type>& v)
        : v_(v)
    {
        // opzionale: verifiche su norma(v) ~ 1
    }

    // -----
    // FASE DI COSTRUZIONE
    // -----
    //
    // Costruisce il riflettore che agisce sul segmento x[begin:begin+len)
    // mandandolo in +/- ||x_segment|| e1 (nel sottospazio).
    //
    // Il vettore v_ memorizzato ha la stessa dimensione di x_segment;
    // in applicazione si suppone di applicare H = I - 2 v v^T
    // all'interno di quel sottospazio, con embedding nel vettore/matrice globale.
    //
    void construct_from_segment(const real_type* x, index_type len)
    {
        assert(x != nullptr);
        assert(len > 0);

        v_.assign(len, real_type{0});
```

```

// Copia x in v_ (per poterlo manipolare)
for (index_type i = 0; i < len; ++i)
    v_[i] = x[i];

// Norma euclidea di x_segment
real_type norm_x2 = 0;
for (index_type i = 0; i < len; ++i)
    norm_x2 += v_[i] * v_[i];

const real_type norm_x = std::sqrt(norm_x2);

if (norm_x == real_type{0}) {
    // Riflettore identità: v = e1 (per convenzione)
    v_[^0] = 1;
    for (index_type i = 1; i < len; ++i) v_[i] = 0;
    return;
}

// Scelta del segno per evitare cancellazione: alpha = sign(x1) * ||x||
const real_type x0 = v_[^0];
const real_type sign = (x0 >= 0) ? 1.0 : -1.0;
const real_type alpha = sign * norm_x;

// u = x + alpha e1
v_[^0] += alpha;

// Ora v_ = u. Normalizziamo per ottenere v = u / ||u||
real_type norm_u2 = 0;
for (index_type i = 0; i < len; ++i)
    norm_u2 += v_[i] * v_[i];

const real_type norm_u = std::sqrt(norm_u2);
if (norm_u == real_type{0}) {
    // Caso patologico (non dovrebbe accadere se norm_x!=0).
    // Prendiamo ancora v = e1.
    v_[^0] = 1;
    for (index_type i = 1; i < len; ++i) v_[i] = 0;
    return;
}

const real_type inv_norm_u = real_type{1} / norm_u;
for (index_type i = 0; i < len; ++i)
    v_[i] *= inv_norm_u;
}

// Accesso al vettore v (solo lettura)
const std::vector<real_type>& vector() const noexcept { return v_; }

// Dimensione del riflettore
index_type size() const noexcept { return v_.size(); }

// -----
// FASE DI APPLICAZIONE
// -----

// Applica  $H = I - 2 v v^T$  al vettore y (segmento in-place).
// Si assume che y_segment corrisponda allo stesso sottospazio
// usato in construct_from_segment (stessa dimensione len).

```

```

void apply_to_segment(real_type* y, index_type len) const
{
    assert(y != nullptr);
    assert(len == v_.size());

    //  $\beta = v^T y$ 
    real_type beta = 0;
    for (index_type i = 0; i < len; ++i)
        beta += v_[i] * y[i];

    //  $y \leftarrow y - 2 \beta v$ 
    const real_type factor = 2 * beta;
    for (index_type i = 0; i < len; ++i)
        y[i] -= factor * v_[i];
}

// Versione che applica H a sinistra:  $y = H*y$ , su un vettore globale
// dove il riflettore agisce solo sulle componenti  $k..n-1$ .
void apply_left_to_vector(real_type* y, index_type n,
                        index_type offset) const
{
    assert(y != nullptr);
    assert(offset + v_.size() <= n);

    //  $\beta = v^T y_{sub}$ 
    real_type beta = 0;
    for (index_type i = 0; i < v_.size(); ++i)
        beta += v_[i] * y[offset + i];

    const real_type factor = 2 * beta;
    for (index_type i = 0; i < v_.size(); ++i)
        y[offset + i] -= factor * v_[i];
}

// Applicazione  $H*A$  (a sinistra) su matrice memorizzata column-major:
// A ha dimensioni  $m \times n$ , riflettore agisce sulle righe  $offset .. offset+len-1$ .
void apply_left_to_matrix(real_type* A,
                        index_type m, index_type n,
                        index_type row_offset) const
{
    assert(A != nullptr);
    assert(row_offset + v_.size() <= m);

    // Per ogni colonna j:
    for (index_type j = 0; j < n; ++j) {

        //  $\beta = v^T (colonna\_j\_sub)$ 
        real_type beta = 0;
        real_type* col_j = A + j * m;

        for (index_type i = 0; i < v_.size(); ++i)
            beta += v_[i] * col_j[row_offset + i];

        const real_type factor = 2 * beta;

        for (index_type i = 0; i < v_.size(); ++i)
            col_j[row_offset + i] -= factor * v_[i];
    }
}

```

```

}

// Variante per applicazione a destra:  $A \leftarrow A * H^T$  (uguale a  $A * H$ ,  $H$  simmetrica)
// qui riflettore agisce sulle colonne  $col\_offset \dots col\_offset + len - 1$ .
void apply_right_to_matrix(real_type* A,
                           index_type m, index_type n,
                           index_type col_offset) const
{
    assert(A != nullptr);
    assert(col_offset + v_.size() <= n);

    // Per ogni riga i:
    for (index_type i = 0; i < m; ++i) {

        //  $\beta = (row\_i\_sub) * v = \sum_k A[i, col\_offset + k] * v[k]$ 
        real_type beta = 0;

        for (index_type k = 0; k < v_.size(); ++k)
            beta += A[i + (col_offset + k) * m] * v_[k];

        const real_type factor = 2 * beta;

        for (index_type k = 0; k < v_.size(); ++k)
            A[i + (col_offset + k) * m] -= factor * v_[k];
    }
}

private:
    std::vector<real_type> v_; // vettore riflettente (normalizzato)
};

```

Questo rispecchia molto da vicino la struttura “ v, τ ” delle routine LAPACK `xLARFG` (generate) e `xLARF` (apply), solo che qui per semplicità uso il v normalizzato e fisso $\tau = 2$.²⁵⁴²⁵⁵²⁵⁶²⁵⁷

Note didattiche da agganciare al codice

- In `construct_from_segment` si vede direttamente la scelta $\alpha = \text{sign}(x_1)\|x\|$ e la costruzione $u = x + \alpha e_1$, $v = u/\|u\|$, esattamente come nelle note standard e nei testi di analisi numerica.²⁵⁸²⁵⁹²⁶⁰²⁶¹²⁶²
- In `apply_*` la formula $y \leftarrow y - 2(v^T y)v$ è chiaramente riconoscibile in due passi: prodotto scalare, poi outer product scalato.²⁶³²⁶⁴²⁶⁵
- Le versioni `apply_left_to_matrix` e `apply_right_to_matrix` ti permettono di mostrare la differenza tra azione su righe e su colonne (si muove l’offset di riga vs l’offset di colonna), e si leggono come “aggiornamento di tutta la colonna/di tutte le colonne interessate” come nei pseudocodici di QR via Householder.²⁶⁶²⁶⁷²⁶⁸

TO DO

- prendere un vettore casuale,
- costruire il riflettore,
- verificare numericamente che la coda sia nulla e che H sia ortogonale (via $H^T H \approx I$ ricostruito esplicitamente solo per piccole dimensioni).²⁶⁹²⁷⁰²⁷¹²⁷²²⁷³²⁷⁴²⁷⁵

Householder in stile imperativo (C++ con stile ANSI C)

Questa nota didattica presenta il riflettore di Householder separando in modo esplicito la **fase di costruzione** dalla **fase di applicazione**, con codice in C++ scritto in stile imperativo, senza classi e con interfacce a puntatori e dimensioni esplicite.²⁷⁶²⁷⁷²⁷⁸²⁷⁹

Idea matematica

Un riflettore di Householder ha la forma

$$H = I - 2vv^T$$

con $\|v\|_2 = 1$. Applicato a un vettore y , produce

$$Hy = y - 2(v^T y)v$$

che è esattamente la formula da tradurre in codice: prima un prodotto scalare, poi una correzione lungo la direzione v .²⁸⁰²⁸¹²⁸²

Nel contesto della QR, dato un segmento di colonna x , si costruisce v in modo che il riflettore mandi x in un multiplo di e_1 , cioè in un vettore con sola prima componente non nulla nel sottospazio attivo.²⁸³²⁸⁴²⁸⁵

Separazione didattica delle fasi

La struttura consigliata è la seguente:²⁸⁶²⁸⁷²⁸⁸

1. **Costruzione**: da un target x si determina il vettore riflettente v .
2. **Applicazione**: il vettore v viene riusato per agire su vettori o matrici.

Questa separazione è concettualmente la stessa che compare nelle routine LAPACK per la generazione del riflettore (DLARFG) e per la sua applicazione (DLARF), ma scritta in C++ “piatto”.²⁸⁹²⁹⁰²⁹¹

Fase di costruzione del vettore di Householder (fase geometrica)

Dato $x \in \mathbb{R}^m$, occorre generare v tale che il riflettore $H = I - 2vv^T$ mandi x in $\pm\|x\|_2 e_1$ e si usa la costruzione standard.²⁹²²⁹³²⁹⁴

$$\alpha = \text{sign}(x_1)\|x\|_2$$

$$u = x + \alpha e_1$$

$$v = \frac{u}{\|u\|_2}$$

La scelta del segno in α serve a evitare cancellazioni numeriche nella prima componente durante la costruzione del vettore riflettente.²⁹⁵²⁹⁶

Funzione di costruzione

```
#include <cstdlib>
#include <cmath>
#include <cassert>
```

```
// Costruisce il vettore v (dimensione len) a partire da x (segmento len).
// Risultato: v memorizza il vettore riflettente normalizzato.
// Convenzione: se x == 0, prendiamo v = e1.
```

```

void householder_construct_segment(const double* x,
                                  std::size_t    len,
                                  double*        v)
{
    assert(x != nullptr);
    assert(v != nullptr);
    assert(len > 0);

    // Copia x in v per manipolarlo

    std::size_t i;
    for (i = 0; i < len; ++i)
        v[i] = x[i];

    // Norma euclidea di x

    double norm_x2 = 0.0;

    // Riflettore identità (nel sottospazio) -> v = e1

    for (i = 0; i < len; ++i)
        norm_x2 += v[i] * v[i];

    if (norm_x2 == 0.0) {
        v[0] = 1.0;
        for (i = 1; i < len; ++i)
            v[i] = 0.0;
        return;
    }

    const double norm_x = std::sqrt(norm_x2);

    // Scelta del segno: alpha = sign(x0) * ||x||

    const double x0 = v[0];
    const double sign = (x0 >= 0.0) ? 1.0 : -1.0;
    const double alpha = sign * norm_x;

    // u = x + alpha e1 (scriviamo u direttamente in v)

    v[0] += alpha;

    // Norma di u

    double norm_u2 = 0.0;
    for (i = 0; i < len; ++i)
        norm_u2 += v[i] * v[i];

    if (norm_u2 == 0.0) {
        // Caso teoricamente raro: ripieghiamo su e1.

        v[0] = 1.0;
        for (i = 1; i < len; ++i)
            v[i] = 0.0;
        return;
    }
}

```

```

const double norm_u = std::sqrt(norm_u2);
const double inv_norm_u = 1.0 / norm_u;

// v = u / ||u||

for (i = 0; i < len; ++i)
    v[i] *= inv_norm_u;
}

```

Osservazioni didattiche

- La funzione non costruisce mai esplicitamente la matrice H ; costruisce solo il suo parametro essenziale, cioè il vettore v .²⁹⁷²⁹⁸²⁹⁹
- Questo è il punto giusto per discutere perché l'approccio implicito sia migliore dell'espansione esplicita della matrice, sia per costo sia per chiarezza algoritmica.³⁰⁰³⁰¹³⁰²
- In caso di vettore nullo, si adotta la convenzione $v = e_1$, che rende il riflettore equivalente all'identità nel sottospazio attivo.³⁰³³⁰⁴
- Qui si vedono *esplicitamente* le formule teoriche: $\alpha = \text{sign}(x_1)\|x\|$, $u = x + \alpha e_1$, $v = u/\|u\|$.

Fase di applicazione a un vettore

Una volta costruito v , l'applicazione usa direttamente la formula

$$y \leftarrow y - 2(v^T y)v$$

che è il vero cuore computazionale del metodo.³⁰⁵³⁰⁶³⁰⁷

Funzione di applicazione a un segmento

```

// Applica H = I - 2 v v^T ad un segmento di vettore y[0:len).

void householder_apply_to_segment(double*      y,
                                  std::size_t  len,
                                  const double* v)
{
    assert(y != nullptr);
    assert(v != nullptr);
    assert(len > 0);

    std::size_t i;

    // beta = v^T y

    double beta = 0.0;

    for (i = 0; i < len; ++i)
        beta += v[i] * y[i];

    const double factor = 2.0 * beta;

    // y <- y - 2 beta v

    for (i = 0; i < len; ++i)
        y[i] -= factor * v[i];
}

```


Lettura algoritmica

L'algoritmo si legge in due passi molto puliti:³⁰⁸³⁰⁹

1. proiezione scalare di y su v , cioè $\beta = v^T y$;
2. sottrazione della componente riflessa, cioè $y \leftarrow y - 2\beta v$. E' la proiezione lungo v .

Questa è la forma più adatta per far vedere agli studenti che il riflettore non è una “scatola nera”, ma un'operazione elementare composta da prodotto scalare più aggiornamento vettoriale.³¹⁰³¹¹

Applicazione a una matrice

Per la QR si applica il riflettore a sinistra alla matrice, cioè $A \leftarrow HA$, agendo sulle righe del sottoblocco attivo a partire dalla riga del pivot.³¹²³¹³

Versione left: $A \leftarrow HA$

Si assume memorizzazione **column-major** (stile Fortran/LAPACK), dimensioni $m \times n$, riflettore che agisce su righe `row_offset .. row_offset+len-1.`, coerente con le librerie numeriche classiche.³¹⁴³¹⁵

Formula: su ogni colonna j , sostituisci il blocco di righe attive con $A_{sub} \leftarrow A_{sub} - 2v(v^T A_{sub})$.³¹⁶³¹⁷

```
// Applica H = I - 2 v v^T a sinistra: A <- H*A.  
// A è m x n, column-major. Il riflettore agisce sulle righe  
// row_offset .. row_offset+len-1 (sottospazio attivo).
```

```
void householder_apply_left_to_matrix(double*      A,  
                                     std::size_t  m,  
                                     std::size_t  n,  
                                     std::size_t  row_offset,  
                                     std::size_t  len,  
                                     const double* v)  
{  
    assert(A != nullptr);  
    assert(v != nullptr);  
    assert(row_offset + len <= m);  
  
    std::size_t j, i;  
  
    // Loop sulle colonne  
  
    for (j = 0; j < n; ++j) {  
        double* col_j = A + j * m;  
  
        // beta = v^T (sottovettore delle righe attive)  
  
        double beta = 0.0;  
        for (i = 0; i < len; ++i)  
            beta += v[i] * col_j[row_offset + i];  
  
        const double factor = 2.0 * beta;  
  
        // Aggiornamento della parte attiva della colonna  
  
        for (i = 0; i < len; ++i)  
            col_j[row_offset + i] -= factor * v[i];  
    }  
}
```

Questa è la trascrizione diretta del passo interno dell'algoritmo di Householder QR: per ogni colonna dal pivot in poi, proiezione $v^T A_{sub}$ e sottrazione dell'outer product.³¹⁸³¹⁹³²⁰

Versione right: $A \leftarrow AH$

Questa versione è utile quando il riflettore va applicato sulle colonne, con $A \leftarrow AH$ (o AH^T , ma $H^T = H$), ad esempio in riduzioni bilaterali o in contesti diversi dalla QR elementare.³²¹³²²

```
// Applica  $H = I - 2vv^T$  a destra:  $A \leftarrow A*H$ .
//  $A$  è  $m \times n$ , column-major. Il riflettore agisce sulle colonne
// col_offset .. col_offset+len-1.

void householder_apply_right_to_matrix(double*      A,
                                       std::size_t  m,
                                       std::size_t  n,
                                       std::size_t  col_offset,
                                       std::size_t  len,
                                       const double* v)
{
    assert(A != nullptr);
    assert(v != nullptr);
    assert(col_offset + len <= n);

    std::size_t i, k;

    // Loop sulle righe

    for (i = 0; i < m; ++i) {

        // beta = (row_i_sub) * v

        double beta = 0.0;

        for (k = 0; k < len; ++k)
            beta += A[i + (col_offset + k) * m] * v[k];

        const double factor = 2.0 * beta;

        // Aggiornamento delle colonne attive della riga i

        for (k = 0; k < len; ++k)
            A[i + (col_offset + k) * m] -= factor * v[k];
    }
}
```

Esempio completo di test

Il seguente main costruisce il riflettore a partire dal vettore $(4, 3, 0)^T$, lo applica, e verifica che il risultato sia $(-5, 0, 0)^T$ a errore numerico vicino alla precisione macchina, come previsto dalla teoria.³²³³²⁴³²⁵

```
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cassert>

void householder_construct_segment(const double* x,
                                  std::size_t  len,
                                  double*      v);
```

```

void householder_apply_to_segment(double*      y,
                                std::size_t   len,
                                const double* v);

static void print_vector(const char* name,
                        const double* x,
                        std::size_t n)
{
    std::size_t i;
    std::printf("%s = [", name);
    for (i = 0; i < n; ++i) {
        std::printf("% .12f", x[i]);
        if (i + 1 < n) std::printf(", ");
    }
    std::printf("]\n");
}

int main()
{
    const std::size_t n = 3;
    double x[n] = {4.0, 3.0, 0.0};
    double v[n];
    double y[n];

    std::size_t i;
    for (i = 0; i < n; ++i)
        y[i] = x[i];

    householder_construct_segment(x, n, v);
    householder_apply_to_segment(y, n, v);

    print_vector("x", x, n);
    print_vector("v", v, n);
    print_vector("H*x", y, n);

    const double err_tail = std::sqrt(y[1]*y[1] + y[2]*y[2]);
    const double err_norm = std::fabs(std::sqrt(y[0]*y[0] + y[1]*y[1] + y[2]*y[2]) - 5.0);

    std::printf("errore coda    = %.12e\n", err_tail);
    std::printf("errore norma    = %.12e\n", err_norm);

    return 0;
}

```

Cosa ci si aspetta in output

Per il vettore $x = (4, 3, 0)^T$, si ottiene $\|x\|_2 = 5$, quindi il riflettore deve portare il vettore in $(-5, 0, 0)^T$ oppure $(5, 0, 0)^T$ a seconda della convenzione di segno adottata nella costruzione.³²⁶³²⁷³²⁸

Con la convenzione usata qui, il risultato atteso è vicino a

```

x   = [ 4.000000000000,  3.000000000000,  0.000000000000]
v   = [ 0.948683298051,  0.316227766017,  0.000000000000]
H*x = [-5.000000000000,  0.000000000000,  0.000000000000]

```

Le quantità `errore coda` ed `errore norma` devono risultare molto piccole, tipicamente dell'ordine di $1e-15$ o vicino, a seconda della piattaforma e del compilatore.³²⁹³³⁰

Compilazione minima

Un esempio minimale di compilazione può essere:³³¹

```
g++ -O2 -std=c++17 householder_test.cpp -o householder_test
./householder_test
```

Punti didattici da fissare (principio “costruzione vs applicazione”)

- Definiamo la trasformazione a livello matematico: $H = I - 2vv^T$, $Hx = x - 2(v^T x)v$.³³²³³³
- Deriviamo la scelta di v a partire da x (target) come in `householder_construct_segment`.³³⁴³³⁵
- Il riflettore **non si memorizza come matrice**, ma tramite il suo vettore generatore v .³³⁶³³⁷
- La distinzione tra costruzione e applicazione è essenziale, sia teoricamente sia computazionalmente.³³⁸³³⁹
- La formula operativa è sempre la stessa: prodotto scalare più aggiornamento rank-1 del blocco attivo.³⁴⁰³⁴¹³⁴²
- In QR, il sottospazio attivo cambia a ogni passo: il riflettore opera solo sulla coda della colonna corrente.³⁴³³⁴⁴

TO DO main di test che:

- genera un vettore,
- costruisce v con `householder_construct_segment`,
- applica il riflettore e verifica numericamente che il risultato ha solo la prima componente non nulla e norma conservata, così da chiudere il cerchio teoria-codice-verifica.³⁴⁵³⁴⁶³⁴⁷³⁴⁸

Incapsulare queste stesse funzioni in una routine `qr_householder(...)` che, colonna dopo colonna, costruisce il vettore riflettente sulla coda attiva e lo applica al sottoblocco rimanente della matrice.³⁴⁹³⁵⁰³⁵¹

QR completa via Householder

In stile imperativo, molto vicino agli schemi classici delle note su Householder-QR.³⁵²³⁵³³⁵⁴³⁵⁵

- uno schema algoritmico;
- una versione C++ “alla C” che usa le funzioni `householder_construct_segment` e `householder_apply_left_to_matrix` ;
- qualche nota su come confrontarla con la bidiagonalizzazione usata per l’SVD.³⁵⁶³⁵⁷³⁵⁸³⁵⁹

1. Schema algoritmico Householder-QR

Per $A \in \mathbb{R}^{m \times n}$, $m \geq n$, vogliamo $A = QR$ con Q ortogonale e R triangolare superiore.³⁶⁰³⁶¹³⁶²

Algoritmo (colonna per colonna).³⁶³³⁶⁴³⁶⁵

1. Per $k = 0, 1, \dots, n-1$:
 - consideri il **sottoblocco attivo** $A_{k:m-1, k:n-1}$;
 - estrai la **coda della colonna k**: $x = A_{k:m-1, k}$;
 - costruisci il vettore di Householder v_k che manda x in $\pm \|x\| e_1$;
 - applichi il riflettore a sinistra al sottoblocco;
 - memorizzi v_k (tipicamente dentro la stessa matrice A).

Algoritmo colonna per colonna con sottoblocco attivo

Alla iterazione k (con $k = 0, 1, \dots, n-1$), la matrice ha già le prime k colonne in forma triangolare superiore; il resto è ancora “grezzo”.³⁶⁶³⁶⁷

La situazione a step k può essere schematizzata come:

	colonne							
	0	1	2		k	k+1		n-1
-----	-----							
riga 0	*	*	*	...	*	*	...	*
riga 1	0	*	*	...	*	*	...	*
riga 2	0	0	*	...	*	*	...	*
...	.	.	.		.	.		.
riga k	0	0	0	...	[x]	?	...	?
riga k+1	0	0	0	...	[x]	?	...	?
...	.	.	.		.	.		.
riga m-1	0	0	0	...	[x]	?	...	?

Dove:

- il blocco tra parentesi quadre [...] nella colonna k rappresenta il **vettore attivo**

$$x = A_{k:m-1, k},$$

- il rettangolo che parte da riga k , colonna k e arriva in basso a destra è il **sottoblocco attivo**

$$A_{k:m-1, k:n-1}.$$

Più esplicitamente, separando per blocchi:

$$A = \begin{bmatrix} R_{\{k \times k\}} & * \\ 0_{\{(m-k) \times k\}} & A_{\{k:m-1, k:n-1\}} \end{bmatrix}$$

dove $R_{k \times k}$ è già triangolare superiore (prime k colonne “finite”), e $A_{k:m-1, k:n-1}$ è il sottoblocco su cui agiamo alla iterazione k .³⁶⁸³⁶⁹

Passo k :

1. **Estrazione della coda della colonna k** Prendiamo il vettore colonna a partire dalla riga k :

$$x = A_{k:m-1, k}.$$

Graficamente è il segmento verticale marcato $[x]$:

```
riga k   | ... [x]
riga k+1 | ... [x]
...
riga m-1 | ... [x]
```

2. **Costruzione del vettore di Householder v_k** Costruiamo $v_k \in \mathbb{R}^{m-k}$ che manda x in $\pm\|x\|e_1$; questo è il passo di “design” del riflettore:

$$H_k = I - 2v_kv_k^T, \quad H_k x = \pm\|x\|e_1.$$

3. **Applicazione del riflettore al sottoblocco attivo** Appliciamo H_k a sinistra al sottoblocco inferiore destro:

$$A_{k:m-1, k:n-1} \leftarrow H_k A_{k:m-1, k:n-1}.$$

Graficamente, agiamo solo sul rettangolo:

	colonne			
	k	k+1		n-1
riga k	[*]	[*]	...	[*]
riga k+1	[0]	[*]	...	[*]
...	.	.		.
riga m-1	[0]	[*]	...	[*]

~~~~~

sottoblocco attivo

Dopo l'applicazione, la colonna  $k$  nel sottoblocco ha zeri sotto la diagonale; le colonne da  $k+1$  in poi sono state aggiornate coerentemente.<sup>370371372</sup>

4. **Memorizzazione di  $v_k$**

La parte triangolare superiore aggiornata della colonna  $k$  contiene ora l'elemento diagonale di  $R$ , mentre la parte sotto la diagonale può essere usata per memorizzare la **coda** del vettore  $v_k$  (le componenti  $2, \dots, m-k$  nel sottospazio).<sup>373374</sup>

---

### Passo $k+1$ :

Alla iterazione successiva, la struttura della matrice è:

|          | colonne |   |     |   |     |     |     |
|----------|---------|---|-----|---|-----|-----|-----|
|          | 0       | 1 | ... | k | k+1 |     | n-1 |
| riga 0   | *       | * | ... | * | *   | ... | *   |
| riga 1   | 0       | * | ... | * | *   | ... | *   |
| ...      | .       | . |     | . | .   |     | .   |
| riga k   | 0       | 0 | ... | * | *   | ... | *   |
| riga k+1 | 0       | 0 | ... | 0 | [x] | ... | ?   |
| ...      | .       | . |     | . | .   |     | .   |
| riga m-1 | 0       | 0 | ... | 0 | [x] | ... | ?   |

cioè:

- le colonne  $0, \dots, k$  sono ormai in forma “quasi finale”: triangolare superiore nella parte rilevante;
- il nuovo **sottoblocco attivo** è ora

$$A_{k+1:m-1, k+1:n-1},$$

e la nuova **coda** su cui costruire  $v_{k+1}$  è

$$x = A_{k+1:m-1, k+1}.$$

Si ripete lo stesso schema:

1. estrazione di  $x = A_{k+1:m-1, k+1}$ ;
2. costruzione di  $v_{k+1}$ ;
3. applicazione di  $H_{k+1}$  al sottoblocco attivo  $A_{k+1:m-1, k+1:n-1}$ ;
4. memorizzazione della coda di  $v_{k+1}$  sotto la diagonale nella colonna  $k+1$ .<sup>375376377</sup>

Iterando fino a  $k = n - 1$ , il sottoblocco attivo si restringe sempre di più nell'angolo in basso a destra, fino a ridursi a una dimensione triviale; la matrice intera è così trasformata in una triangolare superiore  $R$ , con i vettori  $v_k$  “nascosti” sotto la diagonale e pronti per ricostruire o applicare  $Q$ .<sup>378379380 381382383384385386387388389390391</sup>

**Alla fine:**

- $R$  è la parte superiore di  $A$  modificata;
- $Q$  si ottiene come prodotto  $Q = H_0 H_1 \cdots H_{n-1}$ , dove  $H_k = I - 2v_k v_k^T$  esteso all'identità fuori dal sottospazio attivo.<sup>392393394</sup>

Per uso pratico spesso si memorizzano solo i vettori  $v_k$  e un array di scalari, e si applica  $Q$  implicitamente; ma per il confronto didattico probabilmente ti interessa anche una funzione che costruisce  $Q$  esplicito.<sup>395396</sup>

## 2. QR via Householder in stile imperativo

Assumo:

- $A$  in column-major, dimensioni  $m \times n$ ;
- riutilizzo della matrice  $A$  per contenere:
  - sopra la diagonale:  $R$ ;
  - sotto la diagonale: “code” dei vettori  $v_k$ ;
- un array `tau` opzionale se un domani vuoi passare alla forma LAPACK-like  $H_k = I - \tau_k v_k v_k^T$ ; qui possiamo anche fissare  $\tau_k = 2.0$  dato che  $v_k$  è normalizzato.<sup>397398399</sup>

**Funzioni ausiliarie (già viste)**

Uso le versioni che abbiamo costruito prima:<sup>400401</sup>

```
void householder_construct_segment(const double* x,
                                  std::size_t len,
                                  double* v);

void householder_apply_left_to_matrix(double* A,
                                       std::size_t m,
                                       std::size_t n,
                                       std::size_t row_offset,
                                       std::size_t len,
                                       const double* v);
```

**Routine `qr_householder_inplace`**

```
#include <cstdint>
#include <cmath>
#include <cassert>
```

```

// A: m x n, column-major
// Output:
// - parte triangolare superiore di A contiene R
// - parte strettamente sotto-diagonale di A contiene le "code" dei vettori v_k
// - opzionalmente si può riempire tau[k] = 2.0 (se v è normalizzato)
void qr_householder_inplace(double* A,
                           std::size_t m,
                           std::size_t n)
{
    assert(A != nullptr);
    std::size_t k, i;

    const std::size_t K = (m < n) ? m : n;

    for (k = 0; k < K; ++k) {

        // Dimensione del sottospazio attivo (coda da riga k a m-1)
        const std::size_t len = m - k;

        // Puntatore all'inizio del segmento x = A(k:m-1, k)
        double* col_k = A + k * m;
        double* x      = col_k + k;

        // Costruisci il vettore v di dimensione len
        double vbuf[/ *max_len*/ 1024]; // per didattica: in pratica usare alloc dinamica
        assert(len <= 1024);           // o usare std::vector se permesso

        householder_construct_segment(x, len, vbuf);

        // Applica il riflettore H_k al sottoblocco A(k:m-1, k:n-1)
        // riutilizzando householder_apply_left_to_matrix su un sotto-array.
        // Per farlo, passiamo A_sub = A + k (offset di riga), ma mantenendo
        // la stessa leading dimension m. L'offset di riga interno sarà 0.
        householder_apply_left_to_matrix(A + k, // inizio riga k di colonna 0
                                         m - k, // nuove "m" locali
                                         n - k, // nuove "n" locali
                                         0,     // row_offset relativo al sottoblocco
                                         len,   // = m-k
                                         vbuf);

        // Ora vogliamo memorizzare la coda di vbuf nella colonna k di A
        // secondo la convenzione classica:
        // A(k,k) = norma firmata, A(k+1:m-1,k) = componenti 2..len di v (o simili).
        // Per mantenere un confronto semplice con la teoria,
        // possiamo scrivere:
        //
        // - col_k[k] = valore aggiornato (pivot R)
        // - col_k[k+1..m-1] = vbuf[1..len-1]
        //
        // così la colonna k contiene in basso la coda del vettore di Householder.
        for (i = 1; i < len; ++i)
            col_k[k + i] = vbuf[i];
    }
}

```

Note:

- Ho usato un buffer statico vbuf[~1024] solo per non introdurre std::vector; ovviamente in una versione



reale metteresti un'allocazione dinamica o `std::vector<double> v(len);`;

- La logica è esattamente quella delle note di QR: alla colonna  $k$  memorizzi la parte strettamente sotto-diagonale del vettore riflettente.<sup>402403404</sup>

### 3. Costruzione esplicita di $Q$ (per confronto didattico)

Per confrontarti con il vecchio lavoro empirico e con l'uso nella bidiagonalizzazione può tornare utile **formare  $Q$  esplicito**, anche se in pratica spesso lo si usa solo implicitamente.<sup>405406407</sup>

L'idea standard:<sup>408409</sup>

- dopo `qr_householder_inplace`, la matrice  $A$  contiene:
  - in alto  $R$ ;
  - sotto la diagonale, per ogni colonna  $k$ , la coda di  $v_k$  (con  $v_{k,0} = 1$  per convenzione implicita).
- per ottenere  $Q$ , parti da  $Q = I_m$  e applica **all'indietro** i riflettori memorizzati:  $Q \leftarrow H_{K-1} \cdots H_1 H_0 Q$ .

Esempio di funzione:

```
// Ricostruisce Q esplicito a partire da A modificata da qr_householder_inplace.
// Q: matrice m x m, column-major, da riempire (inizialmente identità).
```

```
void qr_form_Q_from_householder(const double* A,
                                std::size_t m,
                                std::size_t n,
                                double* Q)
{
    assert(A != nullptr);
    assert(Q != nullptr);

    std::size_t i, j, k;
    const std::size_t K = (m < n) ? m : n;

    // Inizializza Q all'identità
    for (j = 0; j < m; ++j) {
        for (i = 0; i < m; ++i)
            Q[i + j * m] = (i == j) ? 1.0 : 0.0;
    }

    // Applica i riflettori H_k in ordine inverso: k = K-1, ..., 0
    for (std::size_t kk = 0; kk < K; ++kk) {
        k = K - 1 - kk; // k discendente

        const std::size_t len = m - k;

        // Ricostruisci il vettore v_k di dimensione len a partire da A
        double vbuf[1024];
        assert(len <= 1024);

        // Convenzione: v_k[0] = 1, v_k[i] = A[k+1+i-1, k] per i>=1
        vbuf[0] = 1.0;
        const double* col_k = A + k * m;

        for (i = 1; i < len; ++i)
            vbuf[i] = col_k[k + i];

        // Applica H_k a sinistra a Q(k:m-1, :)
        // cioè al sottoblocco di Q di dimensione (m-k) x m
        // Per farlo, usiamo householder_apply_left_to_matrix
        householder_apply_left_to_matrix(Q + k, // inizio riga k di Q
                                          m - k, // m locale
                                          vbuf,
                                          len);
    }
}
```

```

        m,      // n locale (tutte le colonne di Q)
        0,      // row_offset relativo
        len,
        vbuf);
    }
}

```

Questa routine è quasi una traduzione diretta degli pseudocodici MATLAB per “apply Q from compact Householder form”, con le dovute semplificazioni (tau=2, v normalizzati).<sup>410411412</sup>

---

#### 4. Micro-test QR (per chiudere il giro)

Puoi ora scrivere un `main` che:

1. costruisci una piccola matrice (anche la stessa usata per le prove di bidiagonalizzazione);
2. chiama `qr_householder_inplace`;
3. ricostruisce Q con `qr_form_Q_from_householder`;
4. verifica numericamente:
  - $A \approx QR$ ;
  - $Q^T Q \approx I$ .

Gli schemi di verifica sono gli stessi che si usano nei test di QR/SVD.<sup>413414415</sup>

---

#### 5. Collegamento con la bidiagonalizzazione SVD

Confrontando questo QR con la tua bidiagonalizzazione Householder per SVD puoi far emergere in modo molto chiaro la struttura comune.<sup>416417418</sup>

- QR: *solo* riflettori a sinistra, colonna dopo colonna, per rendere la matrice triangolare superiore.<sup>419420</sup>
- Bidiagonalizzazione: riflettori a sinistra per annullare la coda della colonna, riflettori a destra per annullare la coda della riga successiva; si alternano sinistra/destra.<sup>421422423</sup>

A livello di codice, la differenza principale è solo la combinazione di `householder_apply_left_to_matrix` e `householder_apply_right_to_matrix` in sequenza, seguendo esattamente lo schema Golub–Kahan.<sup>424425</sup>

<sup>426427428429</sup>

## Applicazione a destra: sottoblocco attivo sulle colonne/righe

Supponi di essere dopo il riflettore sinistro  $U_k$  del passo  $k$  (quello che annulla gli elementi sotto  $a_{kk}$ ); ora vuoi applicare un riflettore destro  $V_k$  per eliminare gli elementi a destra di  $a_{k,k+1}$  nella riga  $k$ .<sup>430431432</sup>

La situazione tipica (per  $m \geq n$ ) dopo il **riflettore sinistro** al passo  $k$  è:

|          | colonne |   |     |   |     |         |     |
|----------|---------|---|-----|---|-----|---------|-----|
|          | 0       | 1 | ... | k | k+1 | k+2     | n-1 |
| -----    | -----   |   |     |   |     |         |     |
| riga 0   | *       | * | ... | * | *   | *       | *   |
| riga 1   | 0       | * | ... | * | *   | *       | *   |
| ...      | .       | . | .   | . | .   | .       | .   |
| riga k   | 0       | 0 | ... | * | [x] | [x] ... | [x] |
| riga k+1 | 0       | 0 | ... | 0 | *   | *       | *   |
| ...      | .       | . | .   | . | .   | .       | .   |
| riga m-1 | 0       | 0 | ... | 0 | *   | *       | *   |

A sinistra della colonna  $k+1$ , la riga  $k$  ha già la forma desiderata; a destra, vogliamo trasformare la parte

$$x = A_{k, k+1:n-1}$$

in un vettore con una sola componente non nulla in posizione  $k+1$ .<sup>433434</sup>

Più schematicamente, separando a blocchi:

```
A =
[ *          *          ]
[ 0_{(k)×(k+1)} *      ]
[ 0      a_{k,k+1}  x_tail ]
[ 0          *      *      ]
```

dove  $x_{\text{tail}}$  è la parte da azzerare con un riflettore applicato **a destra** (cioè sulle colonne).<sup>435436</sup>

### Passo destro al livello $k$

#### 1. Estrazione della coda di riga attiva

Consideriamo la riga  $k$  a partire dalla colonna  $k+1$ :

$$x = A_{k, k+1:n-1}.$$

Graficamente:

```
riga k | ... a_{k,k}  [x]  [x]  ...  [x]
          |          ^
          |          \--- coda da azzerare ---/
```

Questo è il segmento che vogliamo trasformare in  $\pm\|x\|e_1$  nel sottospazio orizzontale.<sup>437438439</sup> **2. Costruzione del vettore di Householder destro  $w_k$**

Costruiamo un vettore  $w_k \in \mathbb{R}^{n-(k+1)}$  (dimensione: numero di colonne da  $k+1$  a  $n-1$ ) tale che il riflettore

$$V_k = I - 2w_k w_k^T$$

soddisfi

$$xV_k = \pm\|x\|e_1.$$

È esattamente la stessa formula di costruzione usata per il riflettore sinistro, ma applicata al vettore riga  $x$  invece che al vettore colonna; in implementazione si usa la stessa routine `householder_construct_segment` ma lavorando

sulle colonne di  $A$  (perché  $A$  è column-major).<sup>440441442</sup> **3. Applicazione del riflettore a destra al sottoblocco attivo**

Ora applichiamo  $V_k$  a destra sul **sottoblocco attivo sulle colonne**:

$$A_{k:m-1, k+1:n-1} \leftarrow A_{k:m-1, k+1:n-1} V_k.$$

Questo significa che: - agiamo su tutte le righe dalla  $k$ -esima in giù; - modifichiamo solo le colonne da  $k+1$  a  $n-1$ , cioè il “rettangolo” a destra del pivot corrente.<sup>443444445</sup>

Graficamente:

|          | colonne |       |       |     |       |
|----------|---------|-------|-------|-----|-------|
|          | k       | k+1   | k+2   | n-1 |       |
| -----    |         |       |       |     |       |
| riga k   | *       | [ * ] | [ * ] | ... | [ * ] |
| riga k+1 | 0       | [ * ] | [ * ] | ... | [ * ] |
| ...      | .       | .     | .     |     | .     |
| riga m-1 | 0       | [ * ] | [ * ] | ... | [ * ] |
| ~~~~~    |         |       |       |     |       |

sottoblocco attivo destro

Dopo l'applicazione:

- la riga  $k$  ha zeri a destra tranne in colonna  $k+1$ ;
- il sottoblocco sottostante è stato aggiornato coerentemente.<sup>446447448</sup>

#### 4. Memorizzazione di $w_k$

In maniera completamente analoga al lato sinistro, si memorizza la **coda** del vettore  $w_k$  nelle posizioni “inutili” della riga  $k$ , cioè nelle colonne  $k+2, \dots, n-1$ , lasciando  $A_{k,k+1}$  come elemento di bidiagonale:<sup>449450451</sup>

- $A_{k,k+1}$  diventa l'elemento della superdiagonale della matrice bidiagonale;
- $A_{k,k+2}, \dots, A_{k,n-1}$  contengono le componenti  $2, \dots$  del vettore riflettente  $w_k$ .

#### Passo successivo (livello $k+1$ )

Dopo aver applicato **sia** il riflettore sinistro  $U_k$  **sia** il riflettore destro  $V_k$  al passo  $k$ , la matrice ha questa struttura:<sup>452453</sup>

|          | colonne |   |     |   |     |     |     |     |
|----------|---------|---|-----|---|-----|-----|-----|-----|
|          | 0       | 1 | ... | k | k+1 | k+2 | n-1 |     |
| <hr/>    |         |   |     |   |     |     |     |     |
| riga 0   | *       | * | ... | * | *   | 0   | ... | 0   |
| riga 1   | 0       | * | ... | * | *   | 0   | ... | 0   |
| ...      | .       | . |     | . | .   | .   |     | .   |
| riga k   | 0       | 0 | ... | * | *   | 0   | ... | 0   |
| riga k+1 | 0       | 0 | ... | 0 | [x] | [x] | ... | [x] |
| ...      | .       | . |     | . | .   | .   |     | .   |
| riga m-1 | 0       | 0 | ... | 0 | [x] | [x] | ... | [x] |

- le prime  $k$  righe e colonne hanno già la forma bidiagonale desiderata nelle loro posizioni pertinenti;
- la prossima **colonna attiva** è la  $k+1$  (per il riflettore sinistro successivo);
- la prossima **riga attiva** sarà la  $k+1$ , a partire dalla colonna  $k+2$  (per il riflettore destro successivo).<sup>454455456</sup>

Il ciclo completo di bidiagonalizzazione Householder (Golub–Kahan) si riassume quindi così:<sup>457458459</sup>

- Al passo  $k$ :
  - riflettore sinistro  $U_k$  su  $A_{k:m-1, k:n-1}$  per annullare la coda di colonna  $k$ ;

- riflettore destro  $V_k$  su  $A_{k:m-1, k+1:n-1}$  per annullare la coda della riga  $k$  (dal colonna  $k+2$  in poi).
2. Il sottoblocco attivo si restringe alla parte  $A_{k+1:m-1, k+2:n-1}$  per il passo successivo.

Iterando, ottieni  $U^T AV = B$  con  $B$  bidiagonale superiore, e i vettori di Householder di sinistra e di destra sono memorizzati rispettivamente sotto la diagonale e sopra la superdiagonale, in maniera del tutto analoga al caso QR/LQ.<sup>460461462463</sup>

Puoi prendere questa sezione e inserirla accanto alla descrizione del QR, così hai i due “disegni” speculari (sinistra e destra) per confrontare direttamente il tuo codice di bidiagonalizzazione con la versione teorico-didattica.

Vediamo un esempio completo di `main QR` (piccola matrice  $3 \times 3/4 \times 3$ ) con stampa di  $A$ ,  $Q$ ,  $R$ , e residui numerici da confrontare con il vecchio codice `SVD_QR` ga’ scritto.

464465466467468469470471

Vediamo un **main completo** che:

- costruisce una  $4 \times 3$  semplice ma non banale;
- fa QR in-place con `qr_householder_inplace`;
- ricostruisce  $Q$  con `qr_form_Q_from_householder`;
- stampa  $A$ ,  $Q$ ,  $R$  e le norme dei residui  $\|A - QR\|_F$  e  $\|Q^T Q - I\|_F$ . <sup>472473474475</sup>

Puoi incollare questo blocco nella tua “dispensa QR” accanto alle funzioni già scritte.

---

## Esempio completo di main QR ( $4 \times 3$ , Householder imperativo)

Assumo che nel tuo progetto tu abbia già definito:

- `householder_construct_segment`
- `householder_apply_left_to_matrix`
- `qr_householder_inplace`
- `qr_form_Q_from_householder`

come nelle sezioni precedenti. <sup>476477</sup>

File di esempio: `qr_householder_test.cpp`

```
#include <cstdio>
#include <stddef>
#include <cmath>
#include <cassert>

// Dichiarazioni delle funzioni (implementazioni in altri file/precedenti)
void householder_construct_segment(const double* x,
                                  std::size_t len,
                                  double* v);

void householder_apply_left_to_matrix(double* A,
                                       std::size_t m,
                                       std::size_t n,
                                       std::size_t row_offset,
                                       std::size_t len,
                                       const double* v);

void qr_householder_inplace(double* A,
                            std::size_t m,
                            std::size_t n);

void qr_form_Q_from_householder(const double* A,
                                std::size_t m,
                                std::size_t n,
                                double* Q);

// Funzioni di utilità per stampare e calcolare norme Frobenius

static void print_matrix(const char* name,
                        const double* A,
                        std::size_t m,
                        std::size_t n)
{
    std::printf("%s =\n", name);
    for (std::size_t i = 0; i < m; ++i) {
        std::printf("  [");
        for (std::size_t j = 0; j < n; ++j) {
```

```

        std::printf(" %.10f", A[i + j * m]);
        if (j + 1 < n) std::printf(",");
    }
    std::printf(" ]\n");
}
std::printf("\n");
}

static double frobenius_norm(const double* A,
                             std::size_t m,
                             std::size_t n)
{
    double sum = 0.0;
    for (std::size_t j = 0; j < n; ++j) {
        for (std::size_t i = 0; i < m; ++i) {
            double a = A[i + j * m];
            sum += a * a;
        }
    }
    return std::sqrt(sum);
}

// C = A * B (A: m×k, B: k×n), column-major
static void matmul(const double* A,
                  const double* B,
                  double* C,
                  std::size_t m,
                  std::size_t k,
                  std::size_t n)
{
    for (std::size_t j = 0; j < n; ++j) {
        for (std::size_t i = 0; i < m; ++i) {
            double sum = 0.0;
            for (std::size_t p = 0; p < k; ++p)
                sum += A[i + p * m] * B[p + j * k];
            C[i + j * m] = sum;
        }
    }
}

// C = A^T * B (A: m×k, B: m×n), result C: k×n
static void matmul_AT_B(const double* A,
                       const double* B,
                       double* C,
                       std::size_t m,
                       std::size_t k,
                       std::size_t n)
{
    for (std::size_t j = 0; j < n; ++j) {
        for (std::size_t p = 0; p < k; ++p) {
            double sum = 0.0;
            for (std::size_t i = 0; i < m; ++i)
                sum += A[i + p * m] * B[i + j * m];
            C[p + j * k] = sum;
        }
    }
}

```

```

// C = A - B (stessa dimensione)
static void mat_subtract(const double* A,
                        const double* B,
                        double* C,
                        std::size_t m,
                        std::size_t n)
{
    for (std::size_t j = 0; j < n; ++j) {
        for (std::size_t i = 0; i < m; ++i) {
            C[i + j * m] = A[i + j * m] - B[i + j * m];
        }
    }
}

int main()
{
    // Matrice di test 4x3 (column-major):
    // A =
    // [ 12  -51   4 ]
    // [  6  167 -68]
    // [-4   24 -41]
    // [-1   1   0 ]
    //
    // (è un classico esempio didattico di QR via Householder)
    const std::size_t m = 4;
    const std::size_t n = 3;

    double A[m * n] = {
        // colonna 0
        12.0,  6.0, -4.0, -1.0,
        // colonna 1
        -51.0, 167.0, 24.0, 1.0,
        // colonna 2
        4.0, -68.0, -41.0, 0.0
    };

    // Copia di A per controllo residuo A - Q*R
    double A_orig[m * n];
    for (std::size_t j = 0; j < n; ++j)
        for (std::size_t i = 0; i < m; ++i)
            A_orig[i + j * m] = A[i + j * m];

    std::printf("=== QR Householder (esempio 4x3) ===\n\n");
    print_matrix("A (originale)", A_orig, m, n);

    // Fattorizzazione in-place: A -> [R; vettori di Householder sotto la diag]
    qr_householder_inplace(A, m, n);

    // Estrai R: parte triangolare superiore di A
    double R[m * n];
    for (std::size_t j = 0; j < n; ++j) {
        for (std::size_t i = 0; i < m; ++i) {
            if (i <= j)
                R[i + j * m] = A[i + j * m];
            else
                R[i + j * m] = 0.0;
        }
    }
}

```



```

// Costruisci Q esplicito (m x m)
double Q[m * m];
qr_form_Q_from_householder(A, m, n, Q);

print_matrix("R (triangolare sup.)", R, m, n);
print_matrix("Q (ortogonale)", Q, m, m);

// Calcola residuo A_orig - Q*R
double QR[m * n];
matmul(Q, R, QR, m, m, n);

double RES[m * n];
mat_subtract(A_orig, QR, RES, m, n);

double norm_A = frobenius_norm(A_orig, m, n);
double norm_RES = frobenius_norm(RES, m, n);

std::printf("||A||_F      = %.12e\n", norm_A);
std::printf("||A - Q*R||_F  = %.12e\n", norm_RES);
std::printf("rel. residuo   = %.12e\n\n", norm_RES / norm_A);

// Verifica ortogonalità: Q^T Q = I
double QTQ[m * m];
matmul_AT_B(Q, Q, QTQ, m, m, m);

// Costruisci identità I
double I[m * m];
for (std::size_t j = 0; j < m; ++j)
    for (std::size_t i = 0; i < m; ++i)
        I[i + j * m] = (i == j) ? 1.0 : 0.0;

double QTQmI[m * m];
mat_subtract(QTQ, I, QTQmI, m, m);

double norm_QTQmI = frobenius_norm(QTQmI, m, m);

std::printf("||Q^T*Q - I||_F = %.12e\n", norm_QTQmI);

return 0;
}

```

## Cosa ti aspetti numericamente

Su questo esempio classico, una implementazione corretta di Householder-QR ti dà:<sup>478479480</sup>

- un residuo relativo  $\|A - QR\|_F / \|A\|_F$  dell'ordine di  $10^{-15}$ – $10^{-14}$ ;
- una devianza da identità  $\|Q^T Q - I\|_F$  dello stesso ordine;
- una  $R$  triangolare superiore con diagonale (nel caso completo teorico) positiva a seconda della convenzione di segno.<sup>481482483</sup>

Questo `main` affiancato al tuo vecchio codice di bidiagonalizzazione/SVD ti permette di:

- verificare che il “cuore” Householder (costruzione + applicazione) stia facendo esattamente quello che pensavi nei tentativi empirici;
- confrontare strutturalmente il pattern “solo sinistra per QR” con “sinistra+destra alternati” per la bidiagonalizzazione Golub–Kahan.<sup>484485486</sup>

Poi vediamo una versione analoga di `main` per la bidiagonalizzazione (stessa matrice, stampa di  $U^T AV$  e residuo rispetto a una bidiagonale teorica), per chiudere il confronto QR/SVD.

487488489490491492493494

## 1. Ricostruzione di $U$ dai riflettori sinistri

Assumo che:

- dopo `bidiag_householder_inplace(A,m,n)` i riflettori sinistri  $u_k$  siano memorizzati come code nelle colonne:
  - per ogni  $k$ , colonna  $k$ , righe  $k+1..m-1$  contengono le componenti 2.. di  $u_k$ ;
  - la prima componente è implicita  $u_k = 1$ .<sup>495496497</sup>

La ricostruzione è analoga a `qr_form_Q_from_householder`.<sup>498499</sup>

```
// U: m x m, column-major
void bidiag_form_U_from_left(const double* A,
                             std::size_t m,
                             std::size_t n,
                             double* U)
{
    assert(A != nullptr);
    assert(U != nullptr);

    std::size_t i, j, k;
    const std::size_t K = n; // numero riflettori sinistri (m>=n)

    // U = I_m
    for (j = 0; j < m; ++j)
        for (i = 0; i < m; ++i)
            U[i + j * m] = (i == j) ? 1.0 : 0.0;

    // Applica i riflettori U_k in ordine inverso
    for (std::size_t kk = 0; kk < K; ++kk) {
        k = K - 1 - kk; // k = K-1, ..., 0

        const std::size_t len = m - k;
        double ubuf[1024];
        assert(len <= 1024);

        // Ricostruisci u_k di dimensione len
        ubuf[0] = 1.0;
        const double* col_k = A + k * m;

        for (i = 1; i < len; ++i)
            ubuf[i] = col_k[k + i];

        // Applica H_k = I - 2 u u^T a sinistra su U(k:m-1, :)
        householder_apply_left_to_matrix(U + k,
                                          m - k, // m locale
                                          m,      // tutte le colonne di U
                                          0,
                                          len,
                                          ubuf);
    }
}
```

Così ottieni  $U = U_0 U_1 \dots U_{n-1}$ , con  $U_k$  applicati in ordine inverso alla identità, come prescritto per le sequenze di Householder.<sup>500501502</sup>

---

## 2. Ricostruzione di $V$ dai riflettori destri

Per i riflettori destri  $V_k$ .<sup>503504505</sup>

- dopo `bidiag_householder_inplace`, ogni riga  $k$  memorizza la coda del vettore  $w_k$  sopra la superdiagonale:

- la superdiagonale  $A_{k,k+1}$  è l'elemento di  $B$ ;
- le posizioni  $A_{k,k+2}, \dots, A_{k,n-1}$  contengono le componenti 2.. di  $w_k$ ;
- la prima componente implicita è  $w_k = 1$ .
- questi riflettori sono stati applicati a **destra** all'originale  $A$ ; per ricostruire  $V$  è più comodo lavorare su  $V$  direttamente come prodotto di Householder applicati a destra.<sup>506507508</sup>

Una via semplice (coerente con la tua `householder_apply_right_to_matrix`) è:

- inizializza  $V = I_n$ ;
- per  $k = K - 2, \dots, 0$  (l'ultimo riflettore destro è al massimo  $k = n - 2$ ):
  - ricostruisci  $w_k$  dalla riga  $k$ ;
  - applica  $V \leftarrow VH_k$  usando `householder_apply_right_to_matrix` su tutte le righe di  $V$ , con sottospazio di colonne `k+1..n-1`.<sup>509510511</sup>

```
// V: n x n, column-major
void bidiag_form_V_from_right(const double* A,
                              std::size_t m,
                              std::size_t n,
                              double* V)
{
    assert(A != nullptr);
    assert(V != nullptr);

    std::size_t i, j, k;

    // V = I_n
    for (j = 0; j < n; ++j)
        for (i = 0; i < n; ++i)
            V[i + j * n] = (i == j) ? 1.0 : 0.0;

    // Riflettori destri esistono per k = 0 .. n-2
    if (n < 2) return;

    const std::size_t K = n - 1;

    // Applica V_k in ordine inverso: k = K-1, ..., 0
    for (std::size_t kk = 0; kk < K; ++kk) {
        k = K - 1 - kk; // k scende da n-2 a 0

        const std::size_t len = n - (k + 1);
        if (len == 0) continue;

        double wbuf[1024];
        assert(len <= 1024);

        // Ricostruisci w_k (dimensione len) dalla riga k di A
        // w_k[0] = 1, w_k[j] = A(k, k+1+j) per j>=1
        wbuf[0] = 1.0;

        for (std::size_t jloc = 1; jloc < len; ++jloc) {
            std::size_t col = k + 1 + jloc;
            const double* col_ptr = A + col * m;
            wbuf[jloc] = col_ptr[k];
        }

        // Applica H_k = I - 2 w w^T a destra di V(:, k+1:n-1)
        householder_apply_right_to_matrix(V,
                                           n, // m locale (V è n x n)
                                           n, // n locale
                                           k+1, n-1);
    }
}
```

```

        k + 1,
        len,
        wbuf);
    }
}

```

Questa funzione costruisce  $V = V_0 V_1 \dots V_{n-2}$ , parallelamente a quanto fatto per  $U$ .<sup>512513514</sup>

---

### 3. Uso nel main di test

Agganci queste due funzioni al **main** di bidiagonalizzazione che avevamo abbozzato:

- chiami `bidiag_householder_inplace(A,m,n)`;
- estrai  $B$  da `diag/superdiag`;
- chiami:

```

bidiag_form_U_from_left(A, m, n, U);
bidiag_form_V_from_right(A, m, n, V);

```

- ricomponi  $UBV^T$  e misuri il residuo.<sup>515516517</sup>

A questo punto hai la **coppia completa**:

- QR via Householder (solo sinistra) + test  $A \approx QR$ ;
- Bidiagonalizzazione via Householder (sinistra+destra alternati) + test  $A \approx UBV^T$ .<sup>518519520521</sup>

Resta solo da armonizzare i dettagli di memorizzazione dei vettori  $u_k, w_k$  tra la tua `bidiag_householder_inplace` e queste due funzioni, ma concettualmente seguono lo schema che hai già adottato per la QR.<sup>522523524525526527528529530531</sup>

## Note

# Note

## Notes

- <sup>1</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)
- <sup>2</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>
- <sup>3</sup><https://en.wikipedia.org/wiki/Bidiagonalization>
- <sup>4</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>
- <sup>5</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>
- <sup>6</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>
- <sup>7</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>
- <sup>8</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>
- <sup>9</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>
- <sup>10</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>
- <sup>11</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)
- <sup>12</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>
- <sup>13</sup><https://en.wikipedia.org/wiki/Bidiagonalization>
- <sup>14</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>
- <sup>15</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>
- <sup>16</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>
- <sup>17</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>
- <sup>18</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)
- <sup>19</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>
- <sup>20</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)
- <sup>21</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)
- <sup>22</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>
- <sup>23</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>
- <sup>24</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>
- <sup>25</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)
- <sup>26</sup>[https://en.wikipedia.org/wiki/Householder\\_transformation](https://en.wikipedia.org/wiki/Householder_transformation)
- <sup>27</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>
- <sup>28</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>
- <sup>29</sup>[https://en.wikipedia.org/wiki/Householder\\_transformation](https://en.wikipedia.org/wiki/Householder_transformation)
- <sup>30</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>
- <sup>31</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>
- <sup>32</sup><https://arxiv.org/pdf/math/0603251.pdf>
- <sup>33</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>
- <sup>34</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>
- <sup>35</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>
- <sup>36</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>
- <sup>37</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>
- <sup>38</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>
- <sup>39</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>
- <sup>40</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>
- <sup>41</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>

<sup>42</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>43</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>44</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>

<sup>45</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>46</sup><https://arxiv.org/pdf/math/0603251.pdf>

<sup>47</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>48</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>49</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>

<sup>50</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>

<sup>51</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>

<sup>52</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>

<sup>53</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>54</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>55</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>56</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>57</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>58</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>59</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>60</sup><https://arxiv.org/pdf/math/0603251.pdf>

<sup>61</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>62</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>63</sup><https://arxiv.org/pdf/math/0603251.pdf>

<sup>64</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>65</sup><https://arxiv.org/pdf/math/0603251.pdf>

<sup>66</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>67</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>

<sup>68</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>

<sup>69</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>70</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>

<sup>71</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>72</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>73</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>74</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>75</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>76</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>77</sup><https://arxiv.org/pdf/math/0603251.pdf>

<sup>78</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>

<sup>79</sup><https://arxiv.org/pdf/math/0603251.pdf>

<sup>80</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>81</sup>[https://en.wikipedia.org/wiki/Householder\\_transformation](https://en.wikipedia.org/wiki/Householder_transformation)

<sup>82</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>83</sup><https://arxiv.org/pdf/math/0603251.pdf>

<sup>84</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>85</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>86</sup><https://en.wikipedia.org/wiki/Bidiagonalization>



<sup>87</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>

<sup>88</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>

<sup>89</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>90</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>

<sup>91</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>92</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>

<sup>93</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>94</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>95</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>96</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>

<sup>97</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>98</sup>[https://en.wikipedia.org/wiki/Householder\\_transformation](https://en.wikipedia.org/wiki/Householder_transformation)

<sup>99</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>

<sup>100</sup><https://arxiv.org/pdf/math/0603251.pdf>

<sup>101</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>

<sup>102</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>103</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>104</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>105</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>106</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>107</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>108</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>

<sup>109</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>110</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>111</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>112</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>

<sup>113</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>114</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>

<sup>115</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>116</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>117</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>

<sup>118</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>119</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>120</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>121</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>

<sup>122</sup><https://polytopes.net/courses/Stat624S08/tutorial4.pdf>

<sup>123</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>124</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>125</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>126</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>127</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>

<sup>128</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>129</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>130</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>131</sup><https://arxiv.org/pdf/math/0603251.pdf>

<sup>132</sup><https://www.geometrictools.com/GTE/Mathematics/SingularValueDecomposition.h>

<sup>133</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>

<sup>134</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>135</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>136</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>137</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>138</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>139</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>

<sup>140</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>141</sup>[http://www.fp.tul.cz/~plesinger/my\\_publications/conference/GAMM\\_SIAM\\_DUS06.pdf](http://www.fp.tul.cz/~plesinger/my_publications/conference/GAMM_SIAM_DUS06.pdf)

<sup>142</sup><https://www.youtube.com/watch?v=2OW5Yi6QOdY>

<sup>143</sup><https://people.inf.ethz.ch/gander/talks/SVDnew.pdf>

<sup>144</sup><https://iris.unica.it/handle/11584/312870>

<sup>145</sup>[https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module\\_SVD\\_Procedures.html](https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module_SVD_Procedures.html)

<sup>146</sup><https://www.scribd.com/document/880365358/ln10>

<sup>147</sup>[https://ejml.org/javadoc/org/ejml/dense/row/decompose/qr/QRDecompositionHouseholderColumn\\_ZDRM.html](https://ejml.org/javadoc/org/ejml/dense/row/decompose/qr/QRDecompositionHouseholderColumn_ZDRM.html)

<sup>148</sup><https://www.datacamp.com/fr/tutorial/qr-decomposition>

<sup>149</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>150</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>151</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>152</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>153</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>

<sup>154</sup><https://www.geometrictools.com/GTE/Mathematics/SingularValueDecomposition.h>

<sup>155</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>156</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>157</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>158</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>159</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>160</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>161</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>162</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>163</sup><https://www.active-analytics.com/blog/householder-bidiagonalization/>

<sup>164</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>

<sup>165</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>166</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>167</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>168</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>169</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>

<sup>170</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>

<sup>171</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>

<sup>172</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>173</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>

<sup>174</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>

<sup>175</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>

<sup>176</sup>[https://en.wikipedia.org/wiki/Householder\\_transformation](https://en.wikipedia.org/wiki/Householder_transformation)

<sup>177</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>178</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>179</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>  
<sup>180</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>181</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>182</sup>[https://en.wikipedia.org/wiki/Householder\\_transformation](https://en.wikipedia.org/wiki/Householder_transformation)  
<sup>183</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>  
<sup>184</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>185</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>186</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>187</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>188</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>189</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>  
<sup>190</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>191</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>192</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>193</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>194</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>  
<sup>195</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>196</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>197</sup><https://polytopes.net/courses/Stat624S08/tutorial4.pdf>  
<sup>198</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>199</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>200</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>201</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>202</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>203</sup><https://polytopes.net/courses/Stat624S08/tutorial4.pdf>  
<sup>204</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>205</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>206</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>207</sup><https://polytopes.net/courses/Stat624S08/tutorial4.pdf>  
<sup>208</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>209</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>210</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>211</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>212</sup>[http://www.fp.tul.cz/~plesinger/my\\_publications/conference/GAMM\\_SIAM\\_DUS06.pdf](http://www.fp.tul.cz/~plesinger/my_publications/conference/GAMM_SIAM_DUS06.pdf)  
<sup>213</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>214</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>215</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>216</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>217</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>218</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>219</sup>[http://www.fp.tul.cz/~plesinger/my\\_publications/conference/GAMM\\_SIAM\\_DUS06.pdf](http://www.fp.tul.cz/~plesinger/my_publications/conference/GAMM_SIAM_DUS06.pdf)  
<sup>220</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>221</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>

<sup>222</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>223</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>224</sup>[http://www.fp.tl.cz/~plesinger/my\\_publications/conference/GAMM\\_SIAM\\_DUS06.pdf](http://www.fp.tl.cz/~plesinger/my_publications/conference/GAMM_SIAM_DUS06.pdf)  
<sup>225</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>226</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>227</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>228</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>229</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>230</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>231</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>  
<sup>232</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>233</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>234</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>235</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>236</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>237</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>238</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>239</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>240</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>241</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>242</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>243</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>244</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>245</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>246</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>247</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>248</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>249</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>250</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>251</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>252</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>253</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>254</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>255</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>256</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>257</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>258</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>259</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>260</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>261</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>  
<sup>262</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>  
<sup>263</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>264</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>265</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>266</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>

<sup>267</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>  
<sup>268</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>269</sup>[https://en.wikipedia.org/wiki/Householder\\_transformation](https://en.wikipedia.org/wiki/Householder_transformation)  
<sup>270</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>271</sup><https://polytopes.net/courses/Stat624S08/tutorial4.pdf>  
<sup>272</sup>[http://www.fp.tul.cz/~plesinger/my\\_publications/conference/GAMM\\_SIAM\\_DUS06.pdf](http://www.fp.tul.cz/~plesinger/my_publications/conference/GAMM_SIAM_DUS06.pdf)  
<sup>273</sup><https://www.youtube.com/watch?v=2OW5Yi6QOdY>  
<sup>274</sup><https://people.inf.ethz.ch/gander/talks/SVDnew.pdf>  
<sup>275</sup><https://iris.unica.it/handle/11584/312870>  
<sup>276</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>277</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>278</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>279</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>280</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>281</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>282</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>283</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>284</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>285</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>286</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>287</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>288</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>289</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>290</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>  
<sup>291</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>292</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>293</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>294</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>295</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>296</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>297</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>298</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>299</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>300</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>301</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>302</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>303</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>304</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>305</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>306</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>307</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>308</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>309</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>310</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>311</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)

<sup>312</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>313</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>314</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>315</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>316</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>  
<sup>317</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>318</sup>[https://en.wikipedia.org/wiki/Householder\\_transformation](https://en.wikipedia.org/wiki/Householder_transformation)  
<sup>319</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>320</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>  
<sup>321</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>322</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>323</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>324</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>325</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>326</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>327</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>328</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>329</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>330</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>331</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>332</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>333</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>334</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>335</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>336</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>337</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>338</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>339</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>340</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>341</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>342</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>343</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>344</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>345</sup><https://polytopes.net/courses/Stat624S08/tutorial4.pdf>  
<sup>346</sup>[http://www.fp.tul.cz/~plesinger/my\\_publications/conference/GAMM\\_SIAM\\_DUS06.pdf](http://www.fp.tul.cz/~plesinger/my_publications/conference/GAMM_SIAM_DUS06.pdf)  
<sup>347</sup><https://www.youtube.com/watch?v=2OW5Yi6QOdY>  
<sup>348</sup><https://people.inf.ethz.ch/gander/talks/SVDnew.pdf>  
<sup>349</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>350</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>351</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>352</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>353</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>354</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>355</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>356</sup><https://en.wikipedia.org/wiki/Bidiagonalization>

<sup>357</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>358</sup><https://www.geometrictools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>359</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>360</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>361</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>362</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>363</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>364</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>365</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>366</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>367</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>368</sup>[https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module\\_SVD\\_Procedures.html](https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module_SVD_Procedures.html)  
<sup>369</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>370</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>371</sup>[https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module\\_SVD\\_Procedures.html](https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module_SVD_Procedures.html)  
<sup>372</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>373</sup>[https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module\\_SVD\\_Procedures.html](https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module_SVD_Procedures.html)  
<sup>374</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>375</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>376</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>377</sup>[https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module\\_SVD\\_Procedures.html](https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module_SVD_Procedures.html)  
<sup>378</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>379</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>380</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>381</sup><https://iris.unica.it/handle/11584/312870>  
<sup>382</sup><https://www.youtube.com/watch?v=2OW5Yi6QOdY>  
<sup>383</sup><https://www.math.ucla.edu/~yanovsky/Teaching/Math151B/handouts/GramSchmidt.pdf>  
<sup>384</sup><https://stackoverflow.com/questions/36637322/qr-decomposition-in-r-forcing-a-positive-diagonal>  
<sup>385</sup><https://www.sciencedirect.com/science/article/pii/S0893965999000282>  
<sup>386</sup><https://graphsearch.epfl.ch/en/concept/305223>  
<sup>387</sup><https://www.cs.utexas.edu/~flame/laff/alaff/chapter03-blocked.html>  
<sup>388</sup><https://www.scribd.com/document/880365358/ln10>  
<sup>389</sup>[https://ejml.org/javadoc/org/ejml/dense/row/decompose/qr/QRDecompositionHouseholderColumn\\_ZDRM.html](https://ejml.org/javadoc/org/ejml/dense/row/decompose/qr/QRDecompositionHouseholderColumn_ZDRM.html)  
<sup>390</sup><https://www.datacamp.com/fr/tutorial/qr-decomposition>  
<sup>391</sup><https://gist.github.com/jakelevi1996/512fd1c1291edb07b4242574151088de>  
<sup>392</sup><https://www.geometrictools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>393</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>394</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>395</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>  
<sup>396</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>397</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>  
<sup>398</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>399</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>400</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>401</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>

<sup>402</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>403</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>404</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>405</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>406</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>  
<sup>407</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>408</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>409</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>410</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>  
<sup>411</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>412</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>413</sup>[https://en.wikipedia.org/wiki/Householder\\_transformation](https://en.wikipedia.org/wiki/Householder_transformation)  
<sup>414</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>415</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>416</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>417</sup><https://polytopes.net/courses/Stat624S08/tutorial4.pdf>  
<sup>418</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>419</sup><https://www.active-analytics.com/blog/householder-bidiagonalization-in-d/>  
<sup>420</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>421</sup><https://polytopes.net/courses/Stat624S08/tutorial4.pdf>  
<sup>422</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>423</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>424</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>425</sup><https://polytopes.net/courses/Stat624S08/tutorial4.pdf>  
<sup>426</sup>[http://www.fp.tu.cz/~plesinger/my\\_publications/conference/GAMM\\_SIAM\\_DUS06.pdf](http://www.fp.tu.cz/~plesinger/my_publications/conference/GAMM_SIAM_DUS06.pdf)  
<sup>427</sup><https://www.youtube.com/watch?v=2OW5Yi6QOdY>  
<sup>428</sup><https://people.inf.ethz.ch/gander/talks/SVDnew.pdf>  
<sup>429</sup><https://iris.unica.it/handle/11584/312870>  
<sup>430</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>431</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>432</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>433</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>434</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>435</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>436</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>437</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>438</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>439</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>440</sup><https://www.active-analytics.com/blog/householder-bidiagonalization-in-d/>  
<sup>441</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>442</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>443</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>444</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>445</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>446</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>



<sup>447</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>448</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>449</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>450</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>451</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>452</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>453</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>454</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>455</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>456</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>457</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>458</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>459</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>460</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>461</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>462</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>463</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>464</sup>[https://en.wikipedia.org/wiki/Householder\\_transformation](https://en.wikipedia.org/wiki/Householder_transformation)  
<sup>465</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>466</sup><https://polytopes.net/courses/Stat624S08/tutorial4.pdf>  
<sup>467</sup>[http://www.fp.tul.cz/~plesinger/my\\_publications/conference/GAMM\\_SIAM\\_DUS06.pdf](http://www.fp.tul.cz/~plesinger/my_publications/conference/GAMM_SIAM_DUS06.pdf)  
<sup>468</sup><https://www.youtube.com/watch?v=2OW5Yi6QOdY>  
<sup>469</sup><https://people.inf.ethz.ch/gander/talks/SVDnew.pdf>  
<sup>470</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>  
<sup>471</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>  
<sup>472</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>473</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>474</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>475</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>476</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>477</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>478</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>479</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>480</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>481</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>482</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>483</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>484</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>  
<sup>485</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>  
<sup>486</sup>[https://en.wikipedia.org/wiki/Householder\\_transformation](https://en.wikipedia.org/wiki/Householder_transformation)  
<sup>487</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>488</sup><https://polytopes.net/courses/Stat624S08/tutorial4.pdf>  
<sup>489</sup>[http://www.fp.tul.cz/~plesinger/my\\_publications/conference/GAMM\\_SIAM\\_DUS06.pdf](http://www.fp.tul.cz/~plesinger/my_publications/conference/GAMM_SIAM_DUS06.pdf)  
<sup>490</sup><https://www.youtube.com/watch?v=2OW5Yi6QOdY>  
<sup>491</sup><https://people.inf.ethz.ch/gander/talks/SVDnew.pdf>

<sup>492</sup><https://iris.unica.it/handle/11584/312870>  
<sup>493</sup>[https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module\\_SVD\\_Procedures.html](https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module_SVD_Procedures.html)  
<sup>494</sup><https://www.scribd.com/document/880365358/ln10>  
<sup>495</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>496</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>497</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>498</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>499</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>500</sup><https://www.netlib.org/utk/people/JackDongarra/etemplates/node198.html>  
<sup>501</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>502</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>503</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>504</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>505</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>506</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>507</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>508</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>509</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>510</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>511</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>512</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>513</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>514</sup><https://drsfenner.org/blog/2016/03/householder-bidiagonalization/>  
<sup>515</sup><https://www.active-analytics.com/blog/householder-bigiag-in-d/>  
<sup>516</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>517</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>518</sup><https://www.geometricktools.com/GTE/Mathematics/SingularValueDecomposition.h>  
<sup>519</sup><https://github.com/emchinn/Bidiagonalization/blob/master/Golub-Kahan.ipynb>  
<sup>520</sup>[https://www.math.iit.edu/~fass/477577\\_Chapter\\_12.pdf](https://www.math.iit.edu/~fass/477577_Chapter_12.pdf)  
<sup>521</sup><https://en.wikipedia.org/wiki/Bidiagonalization>  
<sup>522</sup>[https://en.wikipedia.org/wiki/Householder\\_transformation](https://en.wikipedia.org/wiki/Householder_transformation)  
<sup>523</sup><https://arxiv.org/pdf/math/0603251.pdf>  
<sup>524</sup><https://polytopes.net/courses/Stat624S08/tutorial4.pdf>  
<sup>525</sup>[http://www.fp.tul.cz/~plesinger/my\\_publications/conference/GAMM\\_SIAM\\_DUS06.pdf](http://www.fp.tul.cz/~plesinger/my_publications/conference/GAMM_SIAM_DUS06.pdf)  
<sup>526</sup><https://www.youtube.com/watch?v=2OW5Yi6QOdY>  
<sup>527</sup><https://people.inf.ethz.ch/gander/talks/SVDnew.pdf>  
<sup>528</sup><https://iris.unica.it/handle/11584/312870>  
<sup>529</sup>[https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module\\_SVD\\_Procedures.html](https://pagesperso.locean-ipsl.upmc.fr/terray/statpack2.1/manuals/Module_SVD_Procedures.html)  
<sup>530</sup><https://www.sciencedirect.com/science/article/abs/pii/S0096300306003808>  
<sup>531</sup><https://www.netlib.org/lapack/lawnspdf/lawn209.pdf>